

Dokuz Eylül University
Faculty of Science
Department of Mathematics

MAT 4079 GRADUATION PROJECT REPORT

***MATHEMATICAL MODELING OF
A SUN-TRACKING SIX-LEG
PANEL SYSTEM***

by

Berkcan Ahmet ABİ, Selçuk DEVECİ, Aleyna ÖZDOĞAN

Project Supervisor
Prof. Dr. Başak KARPUZ

Fall, 2025-2026

ABSTRACT

This project presents the mathematical modeling and Python-based simulation of a sun-tracking panel system driven by a six-leg parallel mechanism inspired by the Stewart platform architecture. The objective of the system is to continuously orient the solar panel so that its normal vector remains aligned with the Sun's direction throughout the day, thereby maximizing energy efficiency.

The study begins with the geometric formulation of the platform by defining base hinge points, panel attachment points, and coordinate frames for both the fixed base and the moving panel. Using vector geometry, rotation matrices, trigonometric relations, and kinematic constraints, the spatial orientation of the panel is mathematically described. For each Sun position, the required panel tilt is computed under mechanical limits, and the corresponding leg vectors and leg lengths are obtained from the distance relations between the base and panel attachment points.

The complete kinematic model is implemented in Python and validated through a real-time three-dimensional simulation. The simulation visualizes the dynamic motion of the platform, the continuous adjustment of the panel orientation, and the extension and retraction of the six supporting legs as the system tracks the Sun. The results demonstrate the effectiveness of the proposed model in achieving accurate orientation control and provide a practical computational framework applicable to solar tracking systems, parallel robots, and precision engineering mechanisms.

CONTENTS

	Page
ABSTRACT	ii
LIST OF FIGURES	v
1 Introduction	1
2 Applications of Stewart Platforms	2
2.1 Flight Simulation Systems	2
2.2 Driving Simulators and Vehicle Testing	2
2.3 Telescope and Antenna Positioning Systems	2
2.4 Industrial Robotics and Precision Engineering	3
2.5 Solar Tracking Systems	3
3 Problem Statement	4
4 Mathematical Modeling of The System	5
5 Kinematic Formulations	7
5.1 Coordinate Frames	7
5.2 Geometric Parameters	7
5.3 Determination of Panel Orientation	8

5.4	Transformation of Panel Attachment Points	9
5.5	Mathematical Formulation of the Panel Surface	9
5.6	Leg Vectors and Lengths	10
6	Python-Based Simulation	12
6.1	Libraries and Tools	12
6.2	Simulation Code Structure	12
6.3	Visualization and Animation Results	13
6.4	Simulation Code Listing	14
7	Simulation Results	22
	REFERENCES	23

LIST OF FIGURES

	Page
4.1 Model of the sun-tracking six-leg panel system.	6

CHAPTER 1

INTRODUCTION

Solar energy has become one of the most important renewable energy sources in modern engineering due to its sustainability, environmental benefits, and increasing demand for clean power generation. The efficiency of a solar panel system depends largely on how effectively the panel surface is oriented toward the Sun. Since the position of the Sun changes continuously throughout the day, fixed or limited-axis solar panels cannot fully exploit the available solar radiation, which leads to significant energy losses.

To overcome this limitation, advanced solar tracking mechanisms have been developed to provide continuous and precise orientation control. Among these mechanisms, parallel robotic structures offer high stiffness, accuracy, and load-carrying capacity compared to traditional serial mechanisms. The Stewart platform, a six-degree-of-freedom parallel mechanism, is one of the most widely studied and applied structures in precision positioning systems. Its unique architecture allows for accurate control of both position and orientation through the coordinated motion of six adjustable legs.

This project focuses on the mathematical modeling of a sun-tracking panel system based on a Stewart-platform-inspired six-leg mechanism. The primary objective is to develop a complete kinematic framework that enables the panel to continuously align its normal vector with the direction of the Sun while respecting mechanical constraints. The model is constructed using vector geometry, trigonometric relations, coordinate transformations, and rotation matrices to describe the spatial configuration of the platform and its dynamic motion.

In addition to the theoretical formulation, the developed mathematical model is implemented in a Python environment to validate the system through numerical simulation and visualization. The simulation provides a real-time three-dimensional representation of the platform's behavior, allowing the observation of panel orientation, leg length variations, and overall system performance during Sun tracking. By combining mathematical analysis with computational tools, this study aims to bridge theoretical modeling and practical engineering applications, contributing to the design of efficient solar tracking systems and parallel robotic mechanisms.

CHAPTER 2

APPLICATIONS OF STEWART PLATFORMS

Stewart platform mechanisms are widely used in engineering and scientific systems that require precise positioning, high stiffness, and reliable motion control. Their six-degree-of-freedom structure allows them to perform complex spatial movements with high accuracy and stability, making them suitable for a broad range of applications.

2.1 Flight Simulation Systems

Stewart platforms are extensively employed in flight simulators to provide realistic motion feedback for pilot training. The platform reproduces aircraft movements such as roll, pitch, yaw, heave, surge, and sway, allowing pilots to experience real flight conditions in a controlled environment. The high precision and fast response of the mechanism significantly enhance the quality and effectiveness of simulation-based training.

2.2 Driving Simulators and Vehicle Testing

In automotive engineering, Stewart platforms are used in driving simulators and vehicle dynamics testing systems. These platforms simulate road conditions, acceleration, braking, and steering responses. Such simulations enable engineers to evaluate vehicle performance, driver behavior, and safety features without the risks and costs associated with real-world testing.

2.3 Telescope and Antenna Positioning Systems

Stewart platform mechanisms play a critical role in the precise positioning of telescopes and communication antennas. Their ability to maintain stable orientation while isolating external vibrations is essential for high-resolution astronomical observations and reliable satellite communications. Even minor orientation errors in these systems can lead to significant signal loss or measurement inaccuracies.

2.4 Industrial Robotics and Precision Engineering

In industrial environments, Stewart platforms are preferred for applications requiring high stiffness, accuracy, and vibration isolation. They are used in precision assembly, micro-positioning systems, and manufacturing processes involving sensitive components or high-precision tools. Their parallel structure provides superior load distribution and mechanical stability compared to traditional serial robotic arms.

2.5 Solar Tracking Systems

One of the most promising application areas of Stewart platform mechanisms is solar tracking technology. In advanced solar energy systems, precise control of panel orientation is essential to maximize energy capture. The six-leg configuration allows continuous adjustment of both tilt and orientation of the solar panel in real time, ensuring optimal alignment with the Sun throughout the day.

CHAPTER 3

PROBLEM STATEMENT

In modern solar energy systems, the fundamental challenge is to continuously and accurately orient the solar panel toward the Sun as its position changes throughout the day. Fixed panels or limited-axis tracking systems fail to follow the Sun precisely, leading to significant energy loss.

The main problem addressed in this study is the mathematical modeling and control of the spatial orientation of a solar panel using a six-leg mechanism inspired by the Stewart platform, which possesses six degrees of freedom. The objective is to develop a mathematical framework that enables the panel's normal vector to remain aligned with the Sun direction at all times and to accurately compute the corresponding leg lengths required for this motion.

Within this scope, the following key questions are investigated:

- How should the required panel tilt and orientation be computed based on the Sun position?
- How can the extension or contraction of each of the six legs be determined for a given panel configuration?
- How can physically feasible solutions be identified under mechanical constraints?
- How can the developed mathematical model be validated and visualized in a numerical environment?

Solving these problems enables the transfer of theoretical kinematic modeling into a real-time computational and simulation framework, directly contributing to the design of high-efficiency solar tracking systems.

CHAPTER 4

MATHEMATICAL MODELING OF THE SYSTEM

In this chapter, the geometric structure and kinematic behavior of the sun-tracking system are modeled mathematically. The objective is to determine the spatial orientation of the panel based on the Sun position and to compute the required lengths of the six supporting legs.

First, the geometric configuration of the system is defined by specifying the base hinge points and the attachment points on the moving panel. These points are placed symmetrically to ensure mechanical balance and to simplify the mathematical formulation. A global coordinate frame fixed to the base and a local coordinate frame attached to the panel center are introduced. The transformation between these two frames provides a unified representation of the panel's position and orientation. The geometric structure of the proposed system is illustrated in Figure 4.1.

The orientation of the panel is determined using the Sun position vector. At each time step, the Sun's spatial position is computed and normalized, and the panel's normal vector is aligned with this direction. Mechanical tilt limits are incorporated to exclude physically infeasible configurations.

Using the resulting orientation, the attachment points on the panel are transformed into the global coordinate frame. The leg vectors connecting the base hinge points to the panel attachment points are then obtained. The Euclidean norm of each leg vector yields the corresponding instantaneous leg length. In this way, the complete kinematic behavior of the system is mathematically characterized.

This mathematical model is structured in a form suitable for numerical computation and is validated through the Python-based simulation presented in the following chapter.

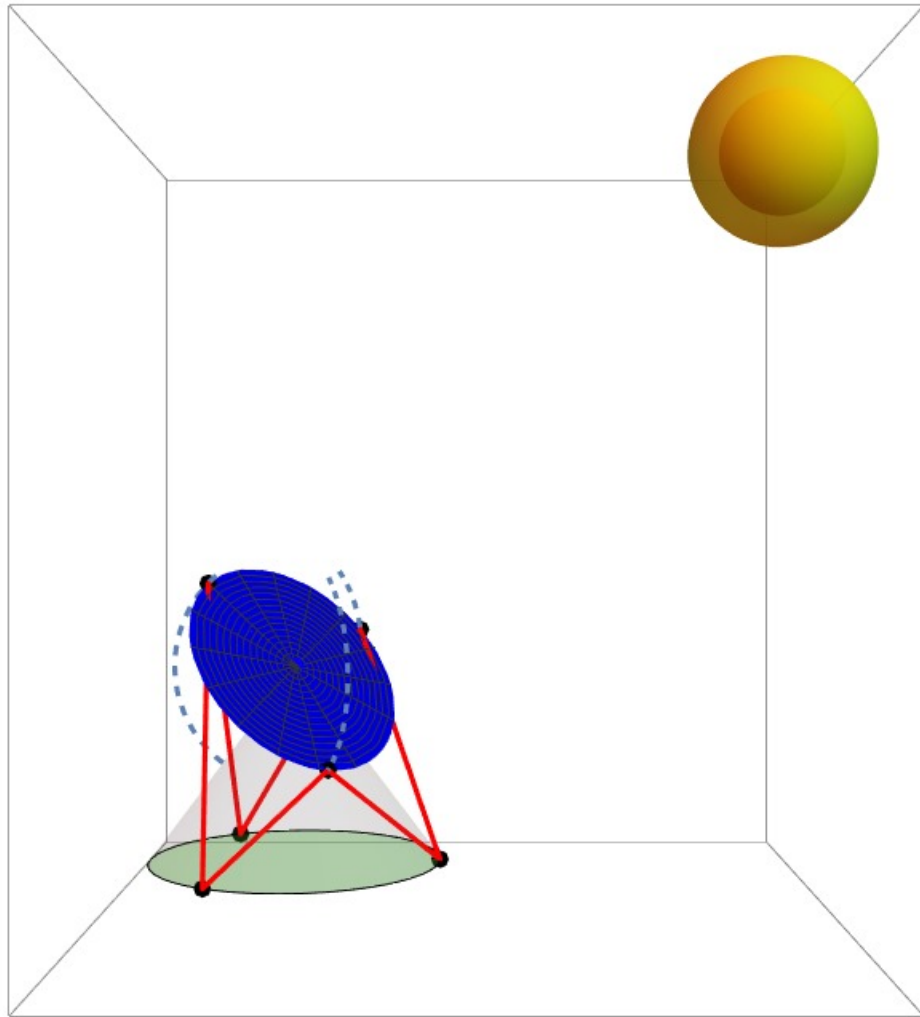


Figure 4.1 Model of the sun-tracking six-leg panel system.

CHAPTER 5

KINEMATIC FORMULATIONS

In this chapter, the kinematic structure of the sun-tracking system is formulated within a rigorous mathematical framework. The objective is to establish explicit relationships between the Sun position, the spatial orientation of the panel, and the corresponding lengths of the six supporting legs. This formulation provides the foundation for the numerical simulation and real-time control of the system.

5.1 Coordinate Frames

Two coordinate frames are introduced in order to describe the geometry and motion of the mechanism. A fixed global coordinate frame $\mathcal{F}_G = \{O; x, y, z\}$ is attached to the center of the base platform, where the z -axis is vertical. A local coordinate frame $\mathcal{F}_P = \{C; x_p, y_p, z_p\}$ is attached to the center of the moving panel, with $C = (0, 0, h)$ representing the nominal height of the panel above the base.

The use of these two frames allows the panel geometry to be defined in a simple form in $\mathcal{F}_P$ and then transformed into $\mathcal{F}_G$ through appropriate rotation and translation operations. This separation greatly simplifies the mathematical modeling and ensures a systematic description of the platform motion.

5.2 Geometric Parameters

The base platform is modeled as a circular disk of radius b_r , while the moving panel is modeled as a circular disk of radius p_r . The Sun position at any time instant is represented by the vector

$$\mathbf{s} = (s_x, s_y, s_z)$$

expressed in the global frame.

The base attachment points are placed symmetrically on the base disk and defined by

$$\mathbf{B}_k = \begin{pmatrix} b_r \cos \alpha_k \\ b_r \sin \alpha_k \\ 0 \end{pmatrix}, \quad \alpha_k = \frac{2\pi k}{3}, \quad k = 0, 1, 2.$$

For the complete six-leg structure, each of these points is duplicated to form pairs of legs, preserving the mechanical symmetry of the system.

The corresponding attachment points on the panel are defined in the local panel frame as

$$P_k(s) = \begin{pmatrix} pr \cos(s) \cos(\beta_k) \\ pr \cos(s) \sin(\beta_k) \\ h + pr \sin(s) \end{pmatrix}, \quad \text{where} \quad \beta_k = \frac{\pi}{3} + \alpha_k, \quad k = 0, 1, 2.$$

5.3 Determination of Panel Orientation

The orientation of the panel is determined from the Sun direction relative to the panel center $C = (0, 0, h)$. Let the direction vector from the panel center to the Sun be defined as:

$$\vec{s}_p = \vec{s} - \vec{C} = (s_x, s_y, s_z - h)$$

In the simulation, the tilt angle s is chosen so that each attachment point $P_k(s)$ lies in the plane orthogonal to $\vec{s}_p$. This condition is expressed by the dot product:

$$(P_k(s) - C) \cdot \vec{s}_p = 0.$$

Expanding this equation using the coordinates of $P_k(s)$ and $\vec{s}_p$:

$$\begin{aligned} (pr \cos s \cos \beta_k) s_x + (pr \cos s \sin \beta_k) s_y + (pr \sin s)(s_z - h) &= 0 \\ pr [\cos s (s_x \cos \beta_k + s_y \sin \beta_k) + \sin s (s_z - h)] &= 0 \end{aligned}$$

Rearranging the terms to isolate the tilt angle s :

$$\begin{aligned}\sin s(s_z - h) &= -\cos s(s_x \cos \beta_k + s_y \sin \beta_k) \\ \frac{\sin s}{\cos s} &= -\frac{s_x \cos \beta_k + s_y \sin \beta_k}{s_z - h}\end{aligned}$$

Thus, the required tilt angle s is computed as:

$$s = \arctan \left(-\frac{s_x \cos \beta_k + s_y \sin \beta_k}{s_z - h} \right).$$

This tilt is constrained by mechanical limitations of the legs, yielding the admissible range:

$$-\arctan \left(\frac{h}{br} \right) \leq s \leq \arctan \left(\frac{h}{br} \right).$$

Any orientation outside this range is considered physically infeasible and is excluded from the simulation.

5.4 Transformation of Panel Attachment Points

Once the tilt angle is determined, the global coordinates of the panel attachment points are obtained by

$$\mathbf{P}_k = P_k(s).$$

This transformation incorporates both the rotation induced by Sun tracking and the constant translation of the panel center above the base.

5.5 Mathematical Formulation of the Panel Surface

The geometry of the solar panel is modeled as a rigid circular disk centered at $C = (0, 0, h)$ with radius pr . To describe points on this surface analytically, a local orthonormal coordinate system is established.

The orientation of the panel is primarily defined by the sun direction. Therefore, the unit

normal vector $\hat{n}$ is set to align with the direction vector $\vec{s}_p$ derived in the previous section:

$$\hat{n} = \frac{\vec{s}_p}{\|\vec{s}_p\|}.$$

To parameterize the circular surface area, an orthonormal basis $\{\hat{u}, \hat{v}\}$ lying within the panel plane is constructed. First, a reference tangent vector is defined using the attachment points P_0 and P_1 :

$$\vec{v}_1 = P_1 - P_0.$$

The first basis vector $\hat{u}$ is aligned with this tangent direction:

$$\hat{u} = \frac{\vec{v}_1}{\|\vec{v}_1\|}.$$

The second basis vector $\hat{v}$ is obtained to be orthogonal to both $\hat{n}$ and $\hat{u}$, ensuring a right-handed coordinate system:

$$\hat{v} = \hat{n} \times \hat{u}.$$

Any point $\vec{X}$ on the panel surface can then be expressed using polar coordinates (r, θ) , where $r \in [0, pr]$ is the radial distance and $\theta \in [0, 2\pi]$ is the azimuth angle. The parametric equation of the surface is given by:

$$\vec{X}(r, \theta) = C + r \cos \theta \hat{u} + r \sin \theta \hat{v}.$$

This formulation maps the planar disk geometry into the 3D spatial coordinates required for the kinematic analysis.

5.6 Leg Vectors and Lengths

The kinematic state of each leg is completely characterized by the leg vector

$$\mathbf{L}_k = \mathbf{P}_k - \mathbf{B}_k,$$

which connects the base hinge point $\mathbf{B}_k$ to the corresponding panel attachment point $\mathbf{P}_k$. The instantaneous length of each leg is then computed as

$$\ell_k = \|\mathbf{L}_k\|.$$

These leg lengths determine the actuator commands required to realize the desired panel orientation. Their continuous variation during Sun tracking reflects the coordinated motion of the six-leg mechanism and ensures precise spatial control of the panel.

CHAPTER 6

PYTHON-BASED SIMULATION

6.1 Libraries and Tools

In this project, Python 3 is used as the primary programming language for implementing the mathematical model of the sun-tracking system and for visualizing its dynamic behavior in three-dimensional space. The main libraries employed in the simulation are:

- **NumPy**: Used for numerical computations, vector and matrix operations, trigonometric functions, and coordinate transformations required by the kinematic model.
- **Matplotlib**: Utilized for three-dimensional plotting and for generating the visual representation of the base platform, panel surface, supporting legs, and the Sun trajectory.
- **FuncAnimation**: A submodule of `matplotlib.animation` that enables real-time frame-by-frame animation of the system.

These tools provide an efficient computational environment for validating the theoretical model and for observing the continuous motion of the sun-tracking mechanism.

6.2 Simulation Code Structure

The simulation is organized using a modular programming approach in order to improve clarity, readability, and maintainability. Each component of the mathematical model is implemented as a separate function, allowing systematic testing and further development.

Geometric and Kinematic Functions

The main functions responsible for the geometric and kinematic computations are:

- `get_bpts()`: Generates the base attachment points of the platform according to the specified geometric parameters.

- `get_ipts(sun_pos)`: Computes the panel attachment points for a given Sun position by determining the required panel orientation and applying the corresponding geometric transformations.
- `get_panel(ipts, sun_pos)`: Constructs the panel surface, determines the panel normal vector, and generates the local coordinate frame of the moving panel.
- `update(frame)`: Executes a complete simulation step by computing the Sun position, updating the panel orientation, recalculating the leg vectors and lengths, and refreshing the three-dimensional visualization.

Sun Motion Model

The apparent motion of the Sun is modeled parametrically over time. At each animation frame, the Sun position vector is updated according to predefined trigonometric relations. This continuous update allows the simulation to represent the daily movement of the Sun and the corresponding response of the tracking mechanism.

Animation Pipeline

- The simulation initializes the three-dimensional scene, including the base disk, panel surface, and supporting legs.
- At each frame, the `update` function recomputes all geometric and kinematic quantities and redraws the scene.
- Leg lengths, panel orientation angles, and system status information are displayed dynamically as on-screen annotations.

6.3 Visualization and Animation Results

The simulation produces a real-time three-dimensional animation of the sun-tracking platform as it follows the Sun's motion. The main visual features of the animation include:

- A fixed base platform representing the foundation of the mechanism.

- A dynamically moving panel surface whose orientation continuously adapts to the changing Sun position.
- Six supporting legs whose lengths vary smoothly according to the computed kinematic relations.
- A visible Sun marker indicating the instantaneous solar direction.

This visual demonstration provides clear evidence of the correctness of the mathematical model and offers an intuitive understanding of how the six-leg mechanism achieves precise sun tracking.

6.4 Simulation Code Listing

The following section presents the complete Python source code used in the simulation. For clarity and readability, the code is divided across multiple pages, with each page containing approximately 15–20 lines.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as animation
4 from mpl_toolkits.mplot3d import Axes3D
5
6 # --- 1. PARAMETERS ---
7 br = 30.0
8 pr = 15.0
9 h = 40.0
10 max_s = np.arctan(h / br)
11
12 # --- BASE POINTS ---
13 def get_bpts():
14     pts = []
15     for k in range(3):
16         angle = k * (2*np.pi) / 3
17         pts.append([br*np.cos(angle), br*np.sin(angle), 0.0])
18     return np.array(pts)

```

```

19
20 #---HINGE POINTS ON PANEL ---
21 def get_ipts(sun_pos):
22
23     sx,sy,sz=sun_pos
24
25     vec_z = sz - h
26     vec_xy = np.sqrt(sx**2 + sy**2)
27
28     current_tilt = np.arctan2(vec_xy, vec_z)
29
30     if current_tilt > max_s:
31         azimuth = np.arctan2(sy, sx)
32
33         new_xy = vec_z * np.tan(max_s)
34         sx = new_xy * np.cos(azimuth)
35         sy = new_xy * np.sin(azimuth)
36
37     ip = []
38     for k in range(3):
39         phi=(np.pi/3)+((k*(2*np.pi))/3)
40         numerator=-(sx*np.cos(phi)+sy*np.sin(phi))
41         denominator=sz-h
42
43         s=np.arctan2(numerator,denominator)
44
45         px=pr*np.cos(s)*np.cos(phi)
46         py=pr*np.cos(s)*np.sin(phi)
47         pz=pr*np.sin(s)+h
48
49         ip.append([px,py,pz])
50
51     return np.array(ip)
52
53 # --- PANEL FRAME ---
54 def get_panel(ipts,sun_pos):

```

```

55     center = np.array([0,0,h])
56     p0, p1, p2 = ipts[0], ipts[1], ipts[2]
57     v1 = p1 - p0
58     v2= p2 - p0
59
60     real_normal=sun_pos-center
61     real_normal/=np.linalg.norm(real_normal)
62
63     normal=np.cross(v1,v2)
64     normal /= np.linalg.norm(normal)
65
66     u = v1 / np.linalg.norm(v1)
67     v = np.cross(normal, u)
68     v /= np.linalg.norm(v)
69
70     r = np.linspace(0, pr, 15)
71     t = np.linspace(0, 2*np.pi, 60)
72     R,T = np.meshgrid(r,t)
73
74     X = center[0] + R*np.cos(T)*u[0] + R*np.sin(T)*v[0]
75     Y = center[1] + R*np.cos(T)*u[1] + R*np.sin(T)*v[1]
76     Z = center[2] + R*np.cos(T)*u[2] + R*np.sin(T)*v[2]
77     return real_normal,normal,X,Y,Z
78
79 # --- SCENE SETUP ---
80 fig = plt.figure(figsize=(13,8))
81 ax = fig.add_subplot(111,projection='3d')
82
83 ax.set_xlim(-100,100)
84 ax.set_ylim(-100,100)
85 ax.set_zlim(0,130)
86
87 # Base Disk
88 theta = np.linspace(0,2*np.pi,60)
89 r = np.linspace(0,br,20)
90 T,R = np.meshgrid(theta,r)

```

```

91 ax.plot_surface(R*np.cos(T), R*np.sin(T), np.zeros_like(R), color='lightgreen',
    alpha=0.25)
92
93 bpts = get_bpts()
94
95 # --- OBJECTS ---
96 sun_scatter = ax.scatter([], [], [], c='orange', s=400, edgecolors='yellow')
97 ipts_scatter = ax.scatter([], [], [], c='black', s=25)
98
99 link_colors = [
100     (1,0,0),
101     (0,1,0),
102     (0,0,1)
103 ]
104
105 leg_lines = []
106 for k in range(3):
107     ln, = ax.plot([], [], [], color=link_colors[k], linewidth=2)
108     leg_lines.append(ln)
109
110 # --- TEXT BLOCKS ---
111 sun_text = ax.text2D(1.05, 0.95, "", transform=ax.transAxes,
112                     fontsize=12, va='top', color='orange')
113
114 leg_texts = []
115 y_positions = [0.88, 0.66, 0.44]
116 for k in range(3):
117     t = ax.text2D(1.05, y_positions[k], "",
118                 transform=ax.transAxes, fontsize=10,
119                 va='top', color=link_colors[k])
120     leg_texts.append(t)
121
122 tilt_text = ax.text2D(1.05, 0.22, "", transform=ax.transAxes,
123                     fontsize=12, va='top', color='Red')
124
125 panel_surf = None

```

```

126
127 # --- UPDATE FUNCTION ---
128 def update(frame):
129     global panel_surf
130
131     total = 200
132     ang = (frame-total/2)/(total/2)
133
134     # SUN
135     sx = -120*np.sin(ang)
136     sy = -80*np.sin(ang)
137     sz = 120*np.cos(ang)
138     sun = np.array([sx,sy,sz])
139
140     ipt_s = get_ipts(sun)
141
142     real_normal, normal,PX, PY, PZ = get_panel(ipts, sun)
143
144
145
146     required_deg=np.degrees(np.arccos(real_normal[2]))
147     tilt_deg = np.degrees(np.arccos(normal[2]))
148     max_tilt_deg = np.degrees(max_s)
149     status = "TRACKING" if required_deg < max_tilt_deg else "OUT OF RANGE"
150
151     if status == "OUT OF RANGE":
152         gray = "gray"
153         panel_color = "gray"
154         tilt_text.set_color(gray)
155         sun_text.set_color(gray)
156         ipt_s.scatter.set_color(gray)
157
158         for k in range(3):
159             leg_texts[k].set_color(gray)
160
161     else:

```

```

162     panel_color = "dodgerblue"
163
164     tilt_text.set_color("red")
165     sun_text.set_color("orange")
166     ipts_scatter.set_color("black")
167
168     for k in range(3):
169         leg_texts[k].set_color(link_colors[k])
170
171 # ---- UPDATE SCATTERS ----
172 sun_scatter._offsets3d = ([sx],[sy],[sz])
173 ipts_scatter._offsets3d = (ipts[:,0], ipts[:,1], ipts[:,2])
174
175 # ---- LEG LINES + TEXT ----
176 for k in range(3):
177     base = bpts[k]
178     panel_pt = ipts[k]
179     next_base = bpts[(k+1) % 3]
180
181     leg_lines[k].set_data(
182         [base[0], panel_pt[0], next_base[0]],
183         [base[1], panel_pt[1], next_base[1]]
184     )
185     leg_lines[k].set_3d_properties(
186         [base[2], panel_pt[2], next_base[2]]
187     )
188
189     right_len = np.linalg.norm(panel_pt - base)
190     left_len = np.linalg.norm(next_base - panel_pt)
191
192     leg_texts[k].set_text(
193         f"Right Leg = {right_len:.2f}\n"
194         f" Base = ({base[0]:6.1f}, {base[1]:6.1f}, {base[2]:4.1f})\n"
195         f" Panel = ({panel_pt[0]:6.1f}, {panel_pt[1]:6.1f}, {panel_pt[2]:4.1f}
196             )\n"
197         f"Left Leg = {left_len:.2f}\n"

```

```

197         f" Panel = ({panel_pt[0]:6.1f}, {panel_pt[1]:6.1f}, {panel_pt[2]:4.1f
           })\n"
198         f" NextBase = ({next_base[0]:6.1f}, {next_base[1]:6.1f}, {next_base
           [2]:4.1f})"
199     )
200
201     # ---- REDRAW PANEL ----
202     if panel_surf is not None:
203         panel_surf.remove()
204
205     panel_surf = ax.plot_surface(PX, PY, PZ, color=panel_color, alpha=1, linewidth
        =0.5)
206
207     # ---- TEXT BLOCK ----
208     tilt_text.set_text(
209         f"Max Zenith: {max_tilt_deg:5.2f}\n"
210         f"Panel Zenith: {tilt_deg:5.2f}\n"
211         f"Required Zenith: {required_deg:5.2f}\n"
212
213
214         f"Status: {status}"
215     )
216
217     sun_text.set_text(
218         f"Sun = ({sx:6.1f}, {sy:6.1f}, {sz:6.1f})"
219     )
220
221     return leg_lines + [sun_scatter, ipts_scatter, panel_surf] + leg_texts + [
        sun_text, tilt_text]
222
223 ani = animation.FuncAnimation(fig, update, frames=200, interval=70)
224 writer = animation.FFMpegWriter(
225     fps=30,
226     metadata=dict(artist='Me'),
227     bitrate=-1,
228     extra_args=['-vcodec', 'libx264', '-crf', '18', '-preset', 'veryslow']

```

```
229 )
230 ani.save('curve_animation.mp4', writer=writer, dpi=300)
231 plt.tight_layout()
232 plt.show()
```

CHAPTER 7

SIMULATION RESULTS

This chapter presents and analyzes the results obtained from the Python-based simulation of the sun-tracking six-leg panel system. The primary objective of the simulation is to evaluate the accuracy of the proposed kinematic model, observe the dynamic behavior of the mechanism, and assess the feasibility of the system under realistic operating conditions.

The simulation demonstrates that the panel is able to continuously adjust its orientation in response to the changing position of the Sun. For each simulation frame, the required panel tilt angle is computed and the corresponding leg lengths are updated accordingly. The results show that the panel's normal vector remains closely aligned with the Sun direction throughout the motion, confirming the correctness of the orientation control strategy.

The computed leg lengths vary smoothly over time, indicating stable and coordinated motion of the six supporting legs. No abrupt changes or discontinuities are observed in the actuator behavior, which is essential for mechanical reliability and control stability. Furthermore, the imposed mechanical constraints effectively prevent the system from entering physically infeasible configurations.

The three-dimensional animation provides an intuitive visualization of the system's performance. The synchronized motion of the panel, supporting legs, and base platform clearly illustrates the effectiveness of the mathematical formulation and the consistency between the theoretical model and numerical implementation.

Overall, the simulation results confirm that the proposed six-leg mechanism can achieve precise and stable sun tracking and that the developed kinematic model provides a robust foundation for the design and optimization of advanced solar tracking systems.

REFERENCES

Tsai, L.-W. (1999). *Robot Analysis: The Mechanics of Serial and Parallel Manipulators*. John Wiley & Sons.

Anton, H., Bivens, I., & Davis, S. (2016). *Calculus: Early Transcendentals* (11th ed.). John Wiley & Sons.

Schaum, D. (1996). *Analytic Geometry*. McGraw-Hill.